

付録 A 目的計算機のアーキテクチャ

A.1 概要

以下のようなレジスタを持つレジスタマシンである。図 22 に構成を示す。

汎用レジスタ： R0, R1, R2, R3

スタックポインタ： SP(R4)

ベースレジスタ： BR(R5)

プログラムカウンタ： PC

コンディションレジスタ： CR

プログラムセグメントレジスタ： PS

データセグメントレジスタ： DS

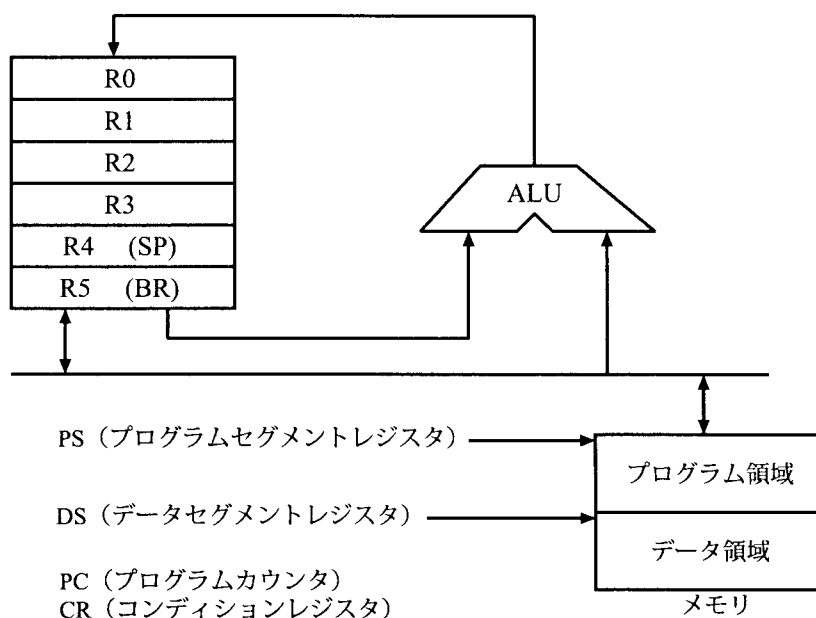


図 22: 目的計算機の構成

各レジスタの大きさは 1word(= 32bit) である。プログラムの実行開始時のレジスタの初期値は次の通りである。

R0,R1,R2,R3 == 不定

SP == -1

BR == 0

PC == 0

CR == 0

PS == 0

DS == 十分大きな値

A.2 命令の語長

各命令の語長は一定で、1word である。命令の種類は次のようなものがある。

命令 (タイプ)	アセンブラ表記		動作	実行サイクル (1 サイクル 1 μ sec.)
Load (type4-1)	load	Rx, addr	$Rx \leftarrow [addr];$	5
Load (type4-2)	load	Rx, offset (BR)	$Rx \leftarrow [BR+offset];$	7
Load (type4-3)	load	Rx, label	$Rx \leftarrow [(label)];$	5
Load register(type3)	loadr	Rx, Ry	$Rx \leftarrow Ry;$	1
Load immediate(type5)	loadi	Rx, immed	$Rx \leftarrow immed;$	3
Store (type4-1)	store	Rx, addr	$[addr] \leftarrow Rx;$	5
Store (type4-2)	store	Rx, offset (BR)	$[BR+offset] \leftarrow Rx;$	7
Store (type4-3)	store	Rx, label	$[(label)] \leftarrow Rx;$	5
Add (type4-1)	add	Rx, addr	$Rx \leftarrow Rx + [addr];$	6
Add (type4-2)	add	Rx, offset (BR)	$Rx \leftarrow Rx + [BR+offset];$	8
Add (type4-3)	add	Rx, label	$Rx \leftarrow Rx + [(label)];$	6
Add register(type3)	addr	Rx, Ry	$Rx \leftarrow Rx + Ry;$	2
Add immediate(type5)	addi	Rx, immed	$Rx \leftarrow Rx + immed;$	4
Subtract (type4-1)	sub	Rx, addr	$Rx \leftarrow Rx - [addr];$	6
Subtract (type4-2)	sub	Rx, offset (BR)	$Rx \leftarrow Rx - [BR+offset];$	8
Subtract (type4-3)	sub	Rx, label	$Rx \leftarrow Rx - [(label)];$	6
Subtract register(type3)	subr	Rx, Ry	$Rx \leftarrow Rx - Ry;$	2
Subtract immediate(type5)	subi	Rx, immed	$Rx \leftarrow Rx - immed;$	4
Multiply (type4-1)	mul	Rx, addr	$Rx \leftarrow Rx * [addr];$	10
Multiply (type4-2)	mul	Rx, offset (BR)	$Rx \leftarrow Rx * [BR+offset];$	12
Multiply (type4-3)	mul	Rx, label	$Rx \leftarrow Rx * [(label)];$	10
Multiply register(type3)	mulr	Rx, Ry	$Rx \leftarrow Rx * Ry;$	6
Multiply immediate(type5)	muli	Rx, immed	$Rx \leftarrow Rx * immed;$	8
Divide (type4-1)	div	Rx, addr	$Rx \leftarrow Rx / [addr];$	10
Divide (type4-2)	div	Rx, offset (BR)	$Rx \leftarrow Rx / [BR+offset];$	12
Divide (type4-3)	div	Rx, label	$Rx \leftarrow Rx / [(label)];$	10
Divide register(type3)	divr	Rx, Ry	$Rx \leftarrow Rx / Ry;$	6
Divide immediate(type5)	divi	Rx, immed	$Rx \leftarrow Rx / immed;$	8
Compare (type4-1)	cmp	Rx, addr	$CR \leftarrow Rx - [addr];$	6
Compare (type4-2)	cmp	Rx, offset (BR)	$CR \leftarrow Rx - [BR+offset];$	8
Compare (type4-3)	cmp	Rx, label	$CR \leftarrow Rx - [(label)];$	6

Compare register(type3)	cmp_r	Rx, Ry	$CR \leftarrow Rx - Ry;$	2
Compare immediate(type5)	cmp_i	Rx, immed	$CR \leftarrow Rx - immed;$	4
Jump (type6)	jmp	label	$PC \leftarrow (label);$	12
Jump Non Zero(type6)	jnz	label	$CR \neq 0$ ならば $PC \leftarrow (label);$	12
Jump Zero(type6)	jz	label	$CR = 0$ ならば $PC \leftarrow (label);$	12
Jump Greater Than(type6)	jgt	label	$CR > 0$ ならば $PC \leftarrow (label);$	12
Jump Greater Equal(type6)	jge	label	$CR \geq 0$ ならば $PC \leftarrow (label);$	12
Jump Less Than(type6)	jlt	label	$CR < 0$ ならば $PC \leftarrow (label);$	12
Jump Less Equal(type6)	jle	label	$CR \leq 0$ ならば $PC \leftarrow (label);$	12
Call(type6)	call	label	$SP \leftarrow SP + 1; [SP] \leftarrow PC;$ $PC \leftarrow (label);$	25
Return(type1)	return		$PC \leftarrow [SP]; SP \leftarrow SP - 1;$	15
Push(type2)	push	Rx	$SP \leftarrow SP + 1; [SP] \leftarrow Rx;$	6
Pop(type2)	pop	Rx	$Rx \leftarrow [SP]; SP \leftarrow SP - 1;$	6
Read decimal(type2)	read	Rx	$Rx \leftarrow$ キーボードから読み込んだ数;	0
Write Character(type2)	writ_ec	Rx	Rx の値を文字として出力;	0
Write Decimal(type2)	writ_ed	Rx	Rx の値を数字として 10 進数で出力;	0
Halt(type1)	halt		マシンの停止;	0

A.5 目的計算機シミュレータの使い方

目的計算機シミュレータのコマンドは次のような形式をしている。

```
% sr input-file
```

input-file は機械語のファイルである。付録 B のアセンブラの出力が使われる。実行の最後に実行にかかった時間が表示される (単位はマイクロ秒)。

付録B アセンブラ

B.1 アセンブラの1行のフォーマット

type1: [label:]opcode
type2: [label:]opcode register
type3: [label:]opcode register register
type4:
 type4-1: [label:]opcode register addr
 type4-2: [label:]opcode register offset (BR)
 type4-3: [label:]opcode register label
type5: [label:]opcode register immed
type6: [label:]opcode label
data: [label:]data value

各項は、スペース、あるいはコンマ(“,”) で区切られなければならない。

label == 任意の識別子

英字で始まり英数字が続くもの(英字には \$, -, . を含む)

例: L01, loop, \$111 など

opcode == 命令のニーモニック

例: load, loadr, addi など(付録Aの命令体系のニーモニックを使う)

register == R0, R1, R2, R3, R4, R5, SP, BR (ただし, R4 = SP, R5 = BR)

‘R’ は小文字でもよい。

addr, offset == 数字(データセグメントのアドレス)

DS + addr, または DS + BR + offset を計算して実効アドレスが決定される。

例: 0, 1, 10, 22 など

immed == 数字あるいは文字。文字はクォーテーションに囲まれたもの。

例: 333, 10, ‘a’, ‘A’, ‘\n’(改行),
 ‘ ’(スペース),
 ‘\’’(シングルクォート),
 ‘\’\’’(バックスラッシュ) など

data == 疑似命令。value を 1word のデータに変換したものに置換される。

value は addr, offset, label あるいは immed (ただし $-2^{31} \leq \text{immed} \leq 2^{31} - 1$)

B.2 アセンブラの使い方

アセンブラコマンドは次のような形式である。

% asm input-file [output-file]

input-file はアセンブリ言語のプログラムである。このファイルをアセンブルして output-file という名前の機械語ファイルを出力する。output-file は省略可能であり、省略された場合は標準出力に機械語が出力される。

付録C 字句解析ルーチン getsym の仕様

ソースファイルは単なる文字の列である。このファイルからコンパイルするための最初の処理は字句解析と呼ばれ、ソース言語の最小の単位を見つける処理を行う。例えば、“100”という3文字は、その文字列の両側が空白の場合は数字の100を表すが、前にアルファベットがあれば変数などの記号(例えば“a100”など)となる。このような違いをきちんと見分けて、ソースファイルの頭から順に、部分的な文字列のある意味のかたまり(記号であるとか数字であるなど)として切り出していくのが字句解析である。

この字句解析ルーチンに関しては、あらかじめこちらで作成した字句解析処理関数を使用してよい。この関数の仕様を以下に示す。

関数名： getsym() 呼び出されるごとにソースファイルから1つの意味ある部分文字列とその意味を返す。

関数への入力：

引数： なし

global 変数： FILE 型の infile という global 変数に、入力ファイルのストリームポインタ (fopen() の返回值) を入れておくこと。

返回值： 関数は値を返さないが、global 変数 tok に1つの単位を切り出した結果が入っている。tok は TOKEN 型の変数であること。TOKEN 型の宣言は次の通りである。

```
typedef struct {
    int attr;
    int value;
    char charvalue[MAXIDLEN + 1];
    int sline;
} TOKEN;
```

ただし MAXIDLEN は識別子の最大文字数を表すマクロ定数である。sline にはそのトークンがあった原始プログラムの行が入っている(エラーメッセージを表示するとき利用できる)。attr には切り出された文字列の種類として、次のようなマクロ定数値が入る。

IDENTIFIER： 変数、プログラム、関数の名前にあたる文字列を表す。そのときの文字列は charvalue に入っている。

NUMBER： 文字列が数字を表しているときこの値が入る。また、実際に数値に変換された値は value に入っている。

RWORD： 文字列が予約語(begin, end, div, var など)の時にこの値が入る。また、それぞれの予約語に対応する以下のマクロ定数値が value に入っている。

```
#define BEGIN      260
#define DIV        261
#define DO         262
#define ELSE       263
#define END        264
#define IF         265
#define PROCEDURE  266
#define PROGRAM    267
#define THEN       268
#define VAR        269
#define WHILE      270
#define READ       271
#define WRITE      272
```

SYMBOL : 文字列が記号 (+,*,<, :=, <=など) の場合, この値が入る。また, それぞれの記号に対応する以下の値が value に入っている。

```
#define PLUS      '+'
#define MINUS     '-'
#define TIMES     '*'
#define LPAREN    '('
#define RPAREN    ')'
#define EQL       '='
#define COMMA     ','
#define PERIOD    '.'
#define SEMICOLON ';'
#define BECOMES   256    /* := */
#define LESSTHAN  '<'
#define LESSEQL   257    /* <= */
#define NOTEQL    258    /* <> */
#define GRTRTHAN  '>'
#define GRTREQL   259    /* >= */
#define COLON     ':'
```

ENDFILE : 入力ファイルの最後に到達したことを表す。

SYMERR : 認識できないトークンが現れたことを表す。

TOKEN の定義と上記の各 define 文のヘッダファイルがあるので, それを利用すること。

使い方 : tok 変数は getsym のプログラムの中で extern 宣言されているので, ユーザで用意すること (単に global 変数 tok を TOKEN 型で宣言すればよい)。また getsym() を使う前に 1 度だけ init_getsym() 関数 (引数なし) を呼んで, getsym() の初期化をしなければならない。プログラムはライブラリとなっているので, 次のようにすれば自動的にリンクされる。

```
% cc user-program.c ... -lics -Llpath -Iipath
```

「-lics」は libics.a をライブラリとして参照するオプションである (libics.a は getsym.c から生成した情報系実験用ライブラリ)。ライブラリ検索パスを指定するオプション-L に, libics.a が存在するディレクトリのパス lpath を与える必要がある。

また, tok 変数の中の値の意味を表すヘッダファイルとして getsym.h がある。getsym を利用するときは, このヘッダファイルを include して使うこと。この中に TOKEN 型の宣言と値の定義が入っている。getsym.h が存在するディレクトリのパス ipath も, インクルードファイル検索パスを指定するオプション-I に与える必要がある。

付録D 演算子順位法のアルゴリズム

D.1 アルゴリズム

“\$” を式の終わりを表す記号とする。また、各演算子の順位関係 ($\ll$) を表す順位関数の作り方は次の D.2 で説明する。

入力: $a_1 a_2 a_3 \dots a_m \$$

処理:

初期設定: スタックに “\$” を積む

以下を繰り返す

- 1) 次の入力記号 a_j を読む
- 2) a_j が演算数 (演算子でない) ならば a_j をスタックに積む
- 3) a_j が演算子ならばスタックの最も上の演算子 a_i との関係によって次のいずれかの処理をする
 - A) $a_i \gg a_j$ である間、以下を繰り返す
 - a_i と演算数をスタックから降ろして出力し、結果の演算数 (格納場所) をスタックに積む
 - B) $a_i \ll a_j$ なら a_j をスタックに積む
 - C) $a_i = a_j$ のとき
 - a_i が “(” なら a_i をスタックから降ろす (その分最上部の演算数をずらす)
 - a_i が “\$” なら終了

D.2 順位関数

$a \ll b$ を表現するためにテーブルを作る方法もあるが、ここでは、作り方が簡単な順位関数を用いる方法を説明する。順位関数は 2 つの関数 f, g からなり、それぞれ記号を引数とし、数値を返す関数である。 f と g が返す数値を元に次のように記号 a, b の順位は決定される。

$$f(a) > g(b) \quad \text{ならば} \quad a \gg b$$

$$f(a) = g(b) \quad \text{ならば} \quad a = b$$

$$f(a) < g(b) \quad \text{ならば} \quad a \ll b$$

f, g の値は次のようにして決める。

2 項演算子に対する優先順位が n 段階あるとする。そのときの順位関数は、各優先順位 $i (i > 0)$ に $2i - 1$ と $2i$ の 2 つの数を割り当てる。そして、その演算子 op が左結合的ならば $f(op) = 2i$, $g(op) = 2i - 1$ とし、右結合的ならば逆の値とする。括弧や i (演算数) や “\$” に対する順位関数は、適当な $m > 2n$ について、 $f("(") = f(i) = m + 1$, $g("(") = g(i) = m$, その他は 0 とする。

この決め方は次のような考え方による。括弧の優先順位はその内部にあるものより小さく、その外部にあるものよりも大きい。 i は “(” と同じと見なす。“(” 全体や i は左から先に演算する。左端の “\$” は “(” と同じ、右端の “\$” は “)” と同じである。

単項演算子 uop については、2 項演算子と異なる記号の時は $g(uop)$ をどの $f(op)$ の値よりも大きな値、 $f(uop)$ を op の優先順位から決まる値にすればよい。“-” のように、2 項演算子と同じ記号が単項演算子として使われるときは、やや面倒である。しかし、“-” が単項演算子として使われる場所は “\$”, “(”, “*”, “div” の直後だけであるので、そこに出てきた “-” を異なる記号として

置き換えて使えばよい。例えば，本実験の場合は，次のようになる。ここで，“!” は単項演算子の “-” である。

	+	-	*	div	!	(	)	i	\$
<i>f</i>	2	2	4	4	6	0	11	11	0
<i>g</i>	1	1	3	3	15	10	0	10	0

付録E 作成実験 1 の簡単な構文解析プログラム例

- メイン関数ファイル

```
#include <stdio.h>
#include <strings.h>
#include <getsym.h>
TOKEN tok;
FILE *infile,*outfile;

void compiler(void);

int main(int argc, char *argv[]){
    fprintf(stderr,"Simple compiler: compile start.\n");
    if(argc != 2){
        fprintf(stderr, "USAGE: comp source-file-name\n");
        exit(1);
    }
    if((infile = fopen(argv[1],"r")) == NULL) {
        fprintf(stderr,"Source file %s can't open.\n",argv[1]);
        exit(1);
    }
    if((outfile = fopen("a.asm","w")) == NULL) {
        fprintf(stderr,"Object file creation failed.\n");
        exit(1);
    }
    compiler();
    fclose(infile);
    fclose(outfile);
    exit(0);
}
```

- コンパイラ本体用ファイル

```
#include <stdio.h>
#include <getsym.h>
extern TOKEN    tok;          /* The current token */
extern FILE     *infile;      /* Source File */
extern FILE     *outfile;     /* Object File */

void error(char *s);
void statement(void);

void compiler(void){
    init_getsym();
    getsym();
    if(tok.attr == RWORD && tok.value == PROGRAM) {
        getsym();
        if(tok.attr == IDENTIFIER) {
            getsym();
            if(tok.attr == SYMBOL && tok.value == SEMICOLON) {
                getsym();
                statement();
                if(tok.attr == SYMBOL && tok.value == PERIOD) {
                    fprintf(stderr, "Parsing Completed. No errors found.\n");
                } else error("At the end, a period is required.");
            } else error("After program name, a semicolon is needed.");
        } else error("Program identifier is needed.");
    } else error("At the first, program declaration is required.");
}

void error(char *s){
    fprintf(stderr, "%s\n", s);
    exit(1);
}

void statement(void){
    /* 式を読み込み、その式を計算し結果を表示するコードを生成する処理 */
}
```